# Hydra

## Configuration Management for ML Experiments

From hardcoded scripts to reproducible, composable pipelines

## What is Hydra?

An open-source framework by Meta for configuring complex Python applications — built specifically for the needs of ML research and experimentation.

**Core promise:** your application logic never changes when you change what you're running.

```
conf/
  config.yaml          ← root config + defaults list
  model/
    logistic_regression.yaml
    random_forest.yaml
  preprocessing/
    ordinal.yaml
    onehot.yaml
  experiment/
    quick_test.yaml
```

Config lives in YAML. Your code reads `cfg`. That's it.

## The Problem Hydra Solves

Every ML project hits the same wall:

| Pain | Symptom |
|---|---|
| Hardcoded params | Edit source to try a new model |
| argparse / Click | `if/elif` chains, parameter pollution |
| Manual tracking | "What params did I use last week?" |
| Reproducibility | Results that can never be recovered |

**The root cause:** configuration is tangled into application code.

Hydra separates them — cleanly and completely.

# Key Concept 1

## Structured Configuration

## Structured Configuration

Config is a **typed, hierarchical object** — not a flat dict of strings.

```yaml
# conf/config.yaml
random_state: 42

model:
  name: logistic_regression
  max_iter: 1000

dataset:
  test_size: 0.2
  stratify: true
```

In your script, you access it like an object:

```python
@hydra.main(version_base=None, config_path="conf", config_name="config")
def main(cfg: DictConfig):
    print(cfg.model.max_iter)   # 1000
    print(cfg.random_state)     # 42
```

**OmegaConf** provides type safety, dot access, and interpolation.

## CLI Overrides

Any config key can be overridden from the command line — no `argparse` needed.

```
# Override a value
uv run python src/v3_hydra_basic.py model.max_iter=500

# Override nested value
uv run python src/v3_hydra_basic.py dataset.test_size=0.3

# Multiple overrides
uv run python src/v3_hydra_basic.py model.max_iter=500 random_state=0
```

The syntax mirrors the YAML structure exactly: `group.key=value`.

Hydra auto-saves the full resolved config alongside every run — so you always know what was used.

# Key Concept 2

## Config Groups

## Config Groups

A config group is a **directory of swappable YAML files** — one per variant.

```
conf/model/
  logistic_regression.yaml   ← default
  random_forest.yaml
  gradient_boosting.yaml
```

The root config declares which group to load by default:

```yaml
# conf/config.yaml
defaults:
  - model: logistic_regression   ← swap this at the CLI
  - preprocessing: ordinal
  - dataset: titanic
  - _self_
```

```
# Swap the entire model config in one word
uv run python src/v4_hydra_groups.py model=random_forest
```

Only the relevant parameters for that model are visible.

# Key Concept 3

## `instantiate()` — Config as Code

## `_target_` and `instantiate()`

This is where Hydra becomes genuinely different.

Add `_target_` to any config group — and Hydra will construct the object for you:

```yaml
# conf/model/random_forest.yaml
_target_: sklearn.ensemble.RandomForestClassifier
n_estimators: 100
max_depth: null
random_state: ${random_state}
```

```python
# Your training script — forever
classifier = hydra.utils.instantiate(cfg.model)
```

`instantiate()` reads `_target_`, imports the class, and passes all other keys as `**kwargs`.

**No `if/elif`. No registry. No dispatch.** The YAML is the object.

## Adding a New Model

With `instantiate()`, extending your experiment is a config problem — not a code problem.

```yaml
# conf/model/svm.yaml   ← new file, no code touched
_target_: sklearn.svm.SVC
kernel: rbf
C: 1.0
random_state: ${random_state}
```

```
uv run python src/v5_hydra_instantiate.py model=svm
```

Works immediately. Zero Python changes.

This pattern scales: same `instantiate()` call handles any sklearn estimator, any feature-engine transformer, or any custom class you write.

# Key Concept 4

## Runtime Overrides & Sweeps

## Ad-hoc Parameters with `+`

Need a kwarg that isn't in your YAML? Append it at runtime with `+`:

```
# warm_start is not in random_forest.yaml — add it on the fly
uv run python src/v5_hydra_instantiate.py \
  model=random_forest \
  +model.warm_start=true

# Add solver and penalty to logistic regression
uv run python src/v5_hydra_instantiate.py \
  model=logistic_regression \
  +model.penalty=l1 \
  +model.solver=saga
```

Because `instantiate()` passes everything as `**kwargs`, any key you add goes straight to the constructor — no code changes required.

`+` appends. `~` removes. Plain `key=value` overrides.

## Multirun Sweeps

Run a grid of configurations with `--multirun`:

```
uv run python src/v6_hydra_advanced.py \
  --multirun model=logistic_regression,random_forest,gradient_boosting
```

Hydra runs each combination and organizes outputs automatically:

```
multirun/2026-04-09_17-45-00/
  model=logistic_regression/  .hydra/config.yaml  metrics.json
  model=random_forest/        .hydra/config.yaml  metrics.json
  model=gradient_boosting/    .hydra/config.yaml  metrics.json
```

Every run's full config is saved alongside its results. Reproducibility by default — not by discipline.

## Experiment Presets

Capture your best configurations as named experiments:

```yaml
# conf/experiment/full_comparison.yaml
# @package _global_
defaults:
  - override /model: random_forest
  - override /preprocessing: mean

model:
  n_estimators: 200
  max_depth: 10
```

```
uv run python src/v6_hydra_advanced.py +experiment=full_comparison
```

Presets compose on top of your defaults — override only what's different. Share them with your team, commit them to git, reference them in papers.

## What We'll See Today

```
v1  Pure Python      All params hardcoded in source
v2  Click CLI        CLI args, but if/elif grows fast
v3  Basic Hydra      Structured config, dot-notation overrides
v4  Config Groups    Swappable components, no irrelevant params
v5  instantiate()    The "aha moment" — config IS the object ✨
v6  Advanced Hydra   Multirun, presets, auto-organized outputs
```

Same Titanic dataset. Same scikit-learn pipeline. Each version strips away one more layer of boilerplate — until v5, where it disappears entirely.

# Let's build it

```
uv sync
uv run python src/v1_pure_python.py
```